

摘要：

英伟达开源机器人操控框架CaP-X，让大模型现场写Python代码控制机器人，成功代码自动存入跨形态技能库。基于该框架的CaP-Agent0在多项任务中成功率追平甚至超越人类专家。英伟达机器人总管Jim Fan称，智能体机器人时代已至。CaP-X标志着机器人领域正式迎来属于自己的“Harness”时代。

Harness（驾驭）的风，终究还是从大模型，吹到了机器人！

刚刚，英伟达开源了一个全新的机器人操控框架——

CaP-X。

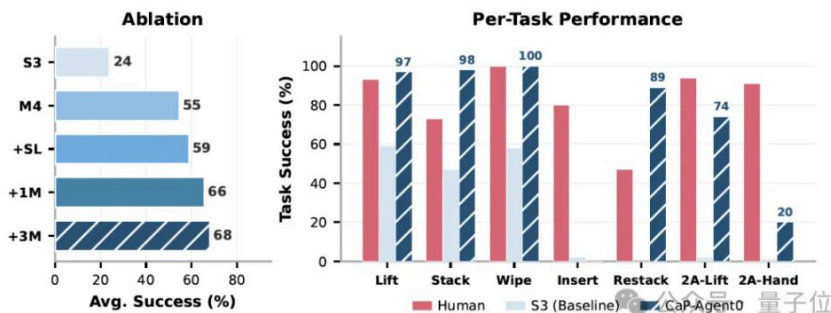
基于这一框架，机器人能盯着摄像头看懂环境，然后现场写一段Python代码来控制自己。

关键，这还不是一次性的。如果某段代码成功完成任务，它会被自动存进技能库，而且适用于不同本体、形态的机器人系统。

（是不是很像龙虾的Skill？）

最离谱的是，这一框架还能把具身大模型（比如VLA）当作API来用，直接一个大脑harness各类小脑（感知与控制）。

在实测表现中，基于这一框架打造的CaP-Agent0，在7项核心任务中，有4项成功率追平甚至超过人类专家手写程序。



即便面对OpenVLA、Pi系列等基于预训练的端到端大模型，CaP-X这种“靠逻辑取胜”的方案也展现出了旗鼓相当、甚至更领先的性能。

Method	libero-object		libero-goal		libero-spatial	
	Pos (Avg.)	Task (Avg.)	Pos (Avg.)	Task (Avg.)	Pos (Avg.)	Task (Avg.)
OpenVLA	0.00	0.00	0.00	0.00	0.00	0.00
π_0	0.00	0.00	0.00	0.00	0.00	0.00
$\pi_{0.5}$	0.17	0.01	0.38	0.00	0.20	0.01
CaP-Agent0	0.22	0.18	0.26	0.17	0.12	0.14

英伟达机器人总管Jim Fan直接下结论：

智能体机器人（Agentic robotics）的时代，来了！



如果说Harness之于大模型，是把引擎装进一辆车；

那么CaP-X之于机器人，就是让这台引擎学会了根据路况自己写驱动程序，并随时给自己升级“代码外挂”。

这一框架的发布，标志着机器人领域正式迎来了属于自己的“Harness”时代。

对此，UCB教授Ken Goldberg评论道：



从“人工脚手架”到“代码即策略”

要理解CaP-X在做什么，咱们先简单回顾一下当前机器人控制的主流做法。

在传统的机器人控制中，工程师得逐行编写感知、规划和反馈逻辑（比如经典的TAMP框架），也就是所谓的Human-in-the-loop（人工干预）。

这种方法虽然精准透明，但泛化性极差——经常是“换个杯子，代码重写”。

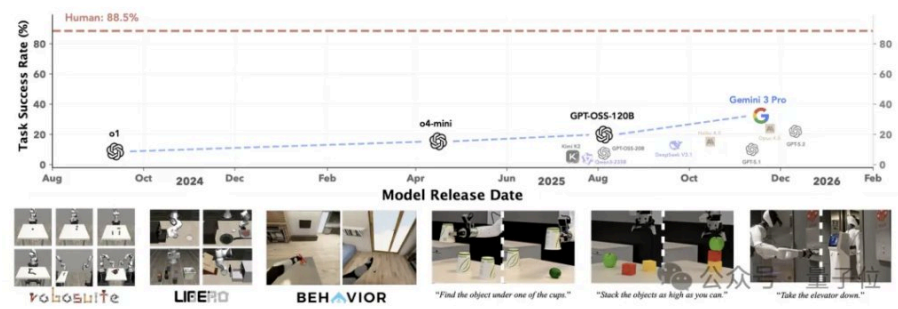
后来，受到大模型Scaling Law的启发，机器人领域开始采用基于数据驱动范式的、端到端的视觉-语言-动作（VLA）模型。

在过去的一年里，VLA 架构（视觉-语言-动作）战绩斐然，机器人开始能叠衣服、干杂活。

但问题在于，VLA是个“黑盒”，一旦出错很难调试，而且遇到新任务还得重新收集数据训练。

也就在最近，受到龙虾（OpenClaw）、Claude code等一系列编程智能体进展的启发。

研究人员开始思考，能否用Gemini、GPT这样的大模型来替代传统控制中工程师的角色，用Python代码直接调用机器人的接口？



而这，就是CaP-X产生的背景，它让大模型从“发号施令的指挥官”，变成了“能写代码的程序员”。

更进一步，在CaP-X框架里，连VLA策略也只是一个可以被随时调用的API。

简单来说，以往的VLA是机器人的“全脑”，从看图像到动手指全靠它。但在CaP-X里，VLA变成了代码里的一行函数。

比如，当机器人需要“把盖子拧开”这种极高频、重手感的精细活时，编程智能体不再自己写复杂的几何坐标，而是直接调用VLA，让VLA来执行精细的复杂操作。

就这样，CaP-X用通用的编程智能体取代了人类工程师，配齐了全套的感知和驱动接口，甚至能在干活的过程中自动合成技能库，调用专攻操作的具身模型。

接下来，我们具体来看。

具身智能的Harness

CaP-X本质上不是一个模型，而是一整套**驾驭框架**，包括：交互式训练环境**CaP-Gym**、层级化基准测试**CaP-Bench**、无需训练的智能体框架**CaP-Agent0**和强化学习进化算法**CaP-RL**。

CaP-Gym

作为整个框架的核心，**CaP-Gym**是一个基于标准Gymnasium接口构建的层级化控制框架。

它将数字大脑和物理身体连接起来，大模型每写出一行代码，物理世界（模拟器或真机）就会实时给出反馈。

在框架上，CaP-Gym统一了感知基元与控制基元：

在感知方面，智能体通过模块化的感知基元从环境中获取数据，这些基元将原始传感器数据抽象为结构化的语义对象。

它内置了**SAM3（语义分割）**和**Molmo 2（点选）**等工具，把原始图像直接变成“这里有一个苹果”、“那里有一个杯子”这种结构化的语义对象。



在控制方面，智能体不直接发布关节空间动作指令，而是调用运动规划器或逆运动学（IK）解算器（如PyRoki）自动处理碰撞检测和路径规划。

也就是说，无论是单手抓取、双臂协作还是移动机器人，CaP-Gym提供了一个让大模型能直接在笛卡尔空间里进行“逻辑编程”的交互式沙盒。

CaP-Bench

在CaP-Gym的基础上，研究还推出了CaP-Bench，用来衡量模型能不能“驾驭”机器人。

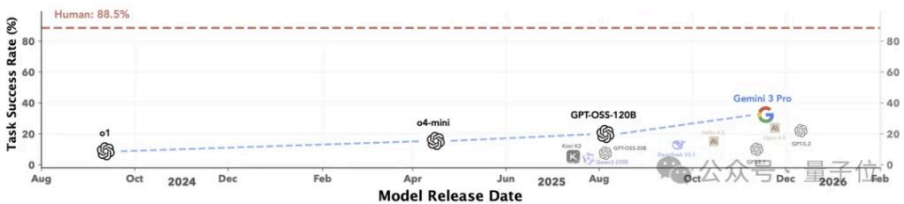
它专门用来测试当模型被推到第一线去“写动作代码”时，它的代码质量、逻辑严密性以及面对物理反馈时的纠错能力到底如何。

CaP-Bench主要从三个维度进行测试：

- 抽象层级（Abstraction Level）： 将动作空间从人工设计的宏命令（高层）转变为原子级的基本基元（底层）；
- 时间交互（Temporal Interaction）： 对比零样本单轮程序生成与多轮交互，以量化故障恢复和迭代推理能力；
- 感知落地（Perceptual Grounding）： 评估不同形式的视觉反馈如何影响智能体将任务相关的视觉特征转化为代码生成的能力。

在对12个最先进的大模型（包括 OpenAI o1、Gemini 3 Pro 等）进行单轮盲测后，结果表明：

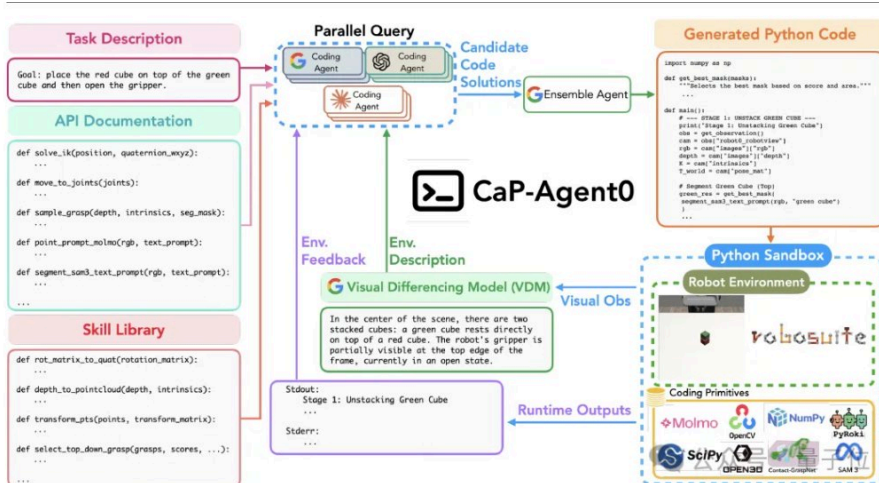
随着人类先验（脚手架）的移除，所有前沿模型的性能断崖式下跌，没有一个能在底层基元上达到人类专家的零样本成功率。



这证明了：如果没有好用的接口，目前强如GPT、Gemini 3 Pro的模型，在底层动作逻辑面前依然会“抓瞎”，离人类专家的水平还差得远。

CaP-Agent0

基于CaP-Bench的失败模式与经验，研究又进一步推出了**CaP-Agent0**。



CaP-Agent0通过专门的多轮推理循环和动态合成的技能库增强了基础模型，核心组件如下：

- 多轮视觉差异比对（VDM）：模型直接看原始图像往往会“瞎”，VDM能将前后帧的视觉差异转化为结构化的自然语言反馈，智能体再基于语言反馈进一步修改代码。
- 自动合成的持久化技能库：当模型在底层瞎摸索偶然成功后，CaP-Agent0会自动提取这段成功的代码，封装成一个可复用的“技能（Skill）”。随着尝试的增多，它自己攒出了一个庞大的技能库，把复杂问题越做越简单。
- 并行集成推理：遇到难题，同时生成多种方案并行尝试，在每一轮中，同时采样多个候选方案。

此外，团队还推出了**CaP-RL**，直接利用环境反馈的成功与否作为可验证奖励，用强化学习（GRPO）来后训练编程模型本身，让它的写码直觉越来越准！

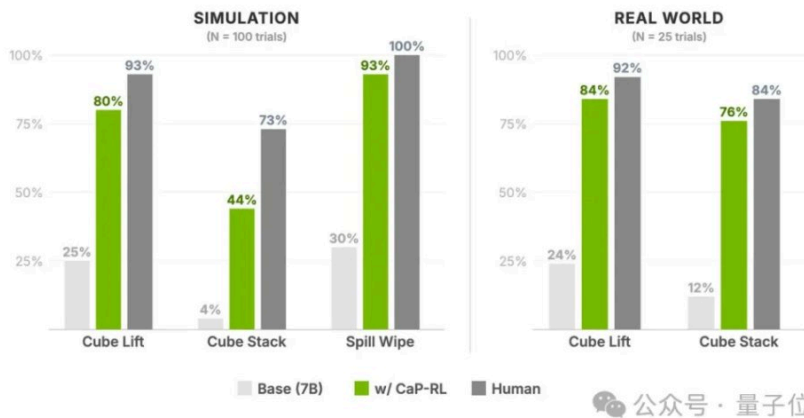
实验结论

如开头所说，在CaP-Bench的7项核心任务中，即便剥离了所有高级接口、只给最底层的原子基元，CaP-Agent0依旧表现优异。

在总共7项任务中，不仅在4项任务上的成功率追平，甚至还**超越了人类专家**写的参考程序。

而在LIBERO-PRO的长程任务中，面对指令或位置的随机干扰，无需训练的CaP-Agent0展现出了比OpenVLA等端到端模型更强大的鲁棒性。

此外，由于CaP-RL是在代码逻辑层进行强化学习，而非像素层，这种能力还可以直接零样本（Zero-shot）无损迁移到真实世界的机器人上。



在论文的最后，团队也坦诚地分享了目前的边界：

虽然程序化控制（CaP）在长程推理和逻辑规划上效果不错，但在需要极高频视觉反馈和细腻触觉感知的“精细活儿”上（比如倒水、精准插拔），目前的纯代码方案还显得有些脆弱。

一个极具前景的方向是**CaP-VLA混合策略**：

由编程智能体管理高层任务逻辑和错误恢复，而将底层执行委派给VLA模型。

从机器人技术的角度来看，通过引入基于优化的控制基元（允许智能体指定任务级约束并考虑避障），可以进一步提升鲁棒性。

代码即策略

实话说，代码即策略（Code as Policy）的思路，其实并不是什么新鲜玩意。



早在2022年，谷歌就提出了CaP。

（是的，Physical Intelligence的CEO Karol Hausman也是当年这篇论文的作者之一。）

CaP的核心思想是：**不要让大模型只输出“下一步做什么”，而是直接让它把机器人执行的Python代码写出来。**

也就是说，不同于以往把大模型当作高层规划器，先输出抽象步骤，再由其他模块执行的方法，CaP直接生成更接近最终控制层的策略代码。

这样做有两个显而易见的好处：

一是代码天然适合表达条件判断、局部反馈回路和精确数值控制。

二是也更容易把“靠近一点”“快一点”这类模糊指令转成具体动作参数。

近年来，沿着这一路径的探索很多，但大多数研究往往难以很好地区分：

到底是模型本身聪明，还是工程师设计的接口提前把活儿给干了一大半？

除此之外，大家也没搞清楚，如果让模型多思考一会儿、多调试几次（也就是测试时计算），能不能弥补它在底层操作上的笨拙。

而CaP-X 就像是这条路线的加强版，它证明了：

具身智能的突破，或许不需要无休止地堆砌真实的遥操数据。

伴随着让模型在推理时多动脑子、用编程智能体代替人类在环的风潮吹向机器人领域，CaP-X给出了一个新解法：

给大模型一个合理的“驾驭框架（Harness）”，让它在代码的虚拟沙箱中自我试错、自我提炼，它就能进化出适应一切物理世界的能力。

CaP-X: A Framework for Benchmarking and Improving Coding Agents for Robot Manipulation

Max Fu^{*1,2} Justin Yu^{*2} Karim El-Refai^{*2} Ethan Kou^{*2} Haoru Xue^{*1,2} Huang Huang³ Wenli Xiao⁴
Guanzhi Wang¹ Fei-Fei Li³ Guanya Shi⁴ Jiajun Wu³ Shankar Sastry² Yuke Zhu¹
Ken Goldberg^{1,2} Linxi “Jim” Fan¹



Ethan Woo @ethancpwoo · 9h

insane scalability with new autonomy stacks for general purpose robotics. Explicitly defining hierarchal properties of long term tasks through LLMs is extremely clever 🤖

极具可扩展性，新增了通用机器人的自主性叠层。通过大型语言模型明确定义长期任务的层级属性非常聪明 🤖



本文来源：[量子位](#)

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

限时 0 元领

* 同一用户免费领取一次

VIP会员 · 7天畅读卡 | 大师课 · 入门串讲

扫码
免费领取



限免



186



收藏

分享到：



写评论

请发表您的评论



表情



图片

发布评论



2 条评论



常规用户

3小时前 中国浙江省

这一框架还能把具身大模型（比如VLA）当作API来用，直接一个大脑harness各类小脑（感知与控制）。。这就像是机器人使用某一套代码框架过程中，感知到某一些方面需要微调，然后就自己更改那部分代码。就跟自己手搓一个自己差不多了。物理意义上的自己手搓一个自己。

👍 0 ↩ 回复



见闻用户

4小时前 中国山西省

资本在毁灭人类的路上渐行渐远

👍 0 ↩ 回复

